TartanGround → LiDAR Pathfinder

Complete Setup & Run Guide

Step-by-step — from raw dataset to trained model to planned path.

Your TartanGround files

- lidar/ .pcd + .ply scans (one per frame)
- imu/ acc.npy, gyro.npy, vel_body.npy, ...
- imu/ pos_global.npy ori_global.npy
- imu/ imu_time.npy parameter.yaml
- image_*cam*/ PNG images (6 cameras)
- image_*_seg/ Semantic segmentation PNGs
- image_*_depth/ Depth PNGs

Files saved by this pipeline

- data/training/traj_*.npz (features+labels+risks)
- models/weights/terrain_classifier.pt
- models/weights/risk_predictor.pt
- models/weights/pointnetpp.pt (optional)
- outputs/paths/<scene>_<traj>_graph.pkl
- outputs/dashboard.png (6-panel analysis)
- outputs/robot_path.gif (animation)
- logs/traversal_events.json (feedback)

IMPORTANT:

If only path needs to be generated using our risk aware algorithm then directly move to last page and run a single command. But make sure that the folder structure is preserved, or it will give file does not exist error.

If model needs to be re trained for weights, then follow the guide below until the last page.
Last page is only to generate the path, and is the only necessary step.

Phase 1 — Install

1 Clone the repository

```
Terminal
git clone https://github.com/your-org/lidar_pathfinder.git
cd lidar_pathfinder
```

2 Install Python packages

```
Terminal
pip install -r requirements.txt
```

NOTE: Takes 3-8 minutes. Installs: numpy scipy scikit-learn torch matplotlib networkx tqdm rich pykdtree yaml

3 Install PCD / PLY reader (required for TartanGround LiDAR)

```
Terminal
pip install open3d
```

IMPORTANT: TartanGround LiDAR files are .pcd and .ply. open3d handles both formats including binary-compressed PCD.

4 Verify everything installed

```
Terminal
python -c "import open3d, numpy, torch, yaml; print('All OK')"
```

EXPECTED OUTPUT

```
All OK
```

Phase 2 — Understand Your Dataset Structure

TartanGround has this exact folder layout. Each scene has many trajectories, each with its own LiDAR, IMU, and images.

Your TartanGround folder structure

```
/mnt/tartanground/                      <-- your dataset root
  forest_01/                            <-- scene name
    P000/                               <-- trajectory ID
      lidar/
        000000.pcd                      <-- one scan per timestep
        000001.pcd
        ...
      image_lcam_front/
        000000.png                      <-- RGB camera frames
      image_lcam_front_seg/
        000000.png                      <-- semantic segmentation
      imu/
        acc.npy          acc.txt
        acc_nograv.npy   acc_nograv.txt
        acc_nograv_body.npy
        gyro.npy         gyro.txt
        pos_global.npy   pos_global.txt
        vel_global.npy   vel_global.txt
        vel_body.npy     vel_body.txt
        ori_global.npy   ori_global.txt
        imu_time.npy     imu_time.txt
        cam_time.npy     cam_time.txt
        parameter.yaml
    P001/
      ...
  urban_02/
    ...
```

5 Inspect your dataset — run this to see all scenes and trajectory counts

inspect_dataset.py -- save and run

```
# inspect_dataset.py
import sys
sys.path.insert(0, '.')
from lidar.tartanground import TartanGroundDataset

ds = TartanGroundDataset('/mnt/tartanground')  # <-- change to your path
ds.print_summary()
```

Terminal

```
python inspect_dataset.py
```

EXPECTED OUTPUT

```
TartanGround Dataset @ /mnt/tartanground
  Scenes: 63
  forest_01                      :  14 trajectories    8,960 lidar frames
  urban_02                       :  12 trajectories    7,680 lidar frames
  desert_rocky_03                :  18 trajectories   11,520 lidar frames
  ...
  TOTAL: 878 trajectories  1,440,000 lidar frames
```

6 Inspect one trajectory — see coordinate ranges for start/goal

inspect_traj.py -- save and run

```
# inspect_traj.py
import sys, numpy as np
sys.path.insert(0, '.')
from lidar.tartanground import TartanGroundDataset
from lidar.preprocessor import estimate_bounds

ds   = TartanGroundDataset('/mnt/tartanground')  # <-- your path
traj = ds.trajectories_in_scene('forest_01')[0]   # first trajectory

print(f'Trajectory: {traj}')
print(f'LiDAR frames: {traj.n_lidar_frames}')

# Load merged cloud (5 frames combined)
cloud  = traj.load_lidar_merged(max_frames=5)
bounds = estimate_bounds(cloud)
print(f'Points (merged): {len(cloud):,}')
print(f'X: {bounds["x_min"]:.1f}  to  {bounds["x_max"]:.1f} m')
print(f'Y: {bounds["y_min"]:.1f}  to  {bounds["y_max"]:.1f} m')
print(f'Z: {bounds["z_min"]:.1f}  to  {bounds["z_max"]:.1f} m')

# IMU summary
imu = traj.load_imu()
print(f'IMU samples: {imu.n_samples}')
print(f'Duration: {imu.duration_s:.1f} s')
print(f'Mean risk (IMU): {imu.risk_from_imu().mean():.3f}')

sx = round(bounds['x_min'] + 2, 1)
sy = round(bounds['y_min'] + 2, 1)
gx = round(bounds['x_max'] - 2, 1)
gy = round(bounds['y_max'] - 2, 1)
print(f'Suggested --start  {sx} {sy} 0')
print(f'Suggested --goal   {gx} {gy} 0')
```

Terminal

```
python inspect_traj.py
```

EXPECTED OUTPUT

```
Trajectory(scene='forest_01', id='P000', lidar_frames=640)
LiDAR frames: 640
Points (merged): 284,291
X: -18.4  to  62.7 m
Y: -5.2   to  71.3 m
Z: -0.3   to  4.8 m
IMU samples: 6,400
Duration: 64.0 s
Mean risk (IMU): 0.312

Suggested --start  -16.4 -3.2 0
Suggested --goal    60.7 69.3 0
```

Phase 3 — Sanity Test (No Training Needed)

Run the full pipeline on one trajectory using heuristic mode (no trained weights). This proves everything works before you spend time training.

7 Run pathfinder on one TartanGround LiDAR frame

Use the coordinates from Step 6

Terminal -- replace scene/traj/frame with your values

```
python main.py \
  --cloud /mnt/tartanground/forest_01/P000/lidar/000000.pcd \
  --start -16.4 -3.2 0 \
  --goal   60.7 69.3 0 \
  --window 5
```

EXPECTED OUTPUT

```
  OK  Loaded 65,536 points from 000000.pcd
  OK  After preprocessing: 12,840 pts  |  Extent 80.2 m
  OK  Features extracted (8 dims per point)
  OK  Terrain labels: ground=7,821  obstacle=3,104  water=412 ...
  OK  Risk (upper bound): mean=0.581  max=0.941
  OK  Graph: 548 nodes (532 traversable)  10,291 edges
  OK  Path found: 24 nodes
      Total dist   : 88.3 m
      Max window risk  : 0.958
```

8 Run with the merged cloud (more points = better path quality)

Merges 8 consecutive LiDAR frames

run_merged.py -- save and run

```
# run_merged.py  --  loads multiple frames and merges into one cloud
import sys, types
sys.path.insert(0, '.')
from lidar.tartanground import TartanGroundDataset
from lidar.preprocessor import preprocess, estimate_bounds
from main import run

ds   = TartanGroundDataset('/mnt/tartanground')  # <-- your path
traj = ds.trajectories_in_scene('forest_01')[0]

# Merge 8 frames into a denser cloud, then save as temp .npy
import numpy as np
cloud = traj.load_lidar_merged(max_frames=8)
np.save('/tmp/merged_cloud.npy', cloud)

bounds = estimate_bounds(cloud)
sx = round(bounds['x_min'] + 2, 1)
sy = round(bounds['y_min'] + 2, 1)
gx = round(bounds['x_max'] - 2, 1)
gy = round(bounds['y_max'] - 2, 1)

args = types.SimpleNamespace(
    cloud='/tmp/merged_cloud.npy', synthetic=False,
    start=[sx, sy, 0.0], goal=[gx, gy, 0.0],
    window=5, voxel=2.0, connect=5.5, max_risk=None,
    weights_clf=None, weights_risk=None,
    no_smooth=False, save_graph=None, load_graph=None,
    alternatives=3, n_synthetic=50000,
    visualize=False, save_anim=None,
)
results = run(args)
```

Terminal

```
python run_merged.py
```

9 Open the dashboard (if you have a display)

Terminal -- opens 6-panel matplotlib plot

```
python main.py \
```

```
  --cloud /tmp/merged_cloud.npy \
  --start -16.4 -3.2 0 \
  --goal   60.7 69.3 0 \
  --window 5 \
  --visualize
```

NOTE: On a headless server, remove --visualize. The path still computes and prints. The dashboard PNG is saved automatically to outputs/dashboard.png.

Phase 4 — Build Training Dataset from IMU

This is the key step that makes TartanGround powerful. The IMU files contain acc, gyro, vel_body, and pos_global — exactly what we need to auto-label terrain risk across all 878 trajectories without any manual annotation.

The pipeline does this per trajectory:

- Load merged LiDAR cloud (multiple .pcd frames)
- Extract terrain features (slope, roughness, curvature, wetness)
- Auto-label terrain class (heuristic: ground / rock / water / vegetation)
- Load IMU files -- compute vibration_signal() and slip_signal()
- For each point, find nearby IMU samples via pos_global and average their risk
- Save features.npy + labels.npy + risks.npy as a .npz training file

10 Run the training dataset builder

Processes all scenes. Adjust max_trajs to how many you want to use.

build_training_data.py -- save and run

```
# build_training_data.py  --  save and run this
import sys
sys.path.insert(0, '.')
from models.feedback_loop import build_imu_risk_dataset

build_imu_risk_dataset(
    dataset_root  = '/mnt/tartanground',   # <-- your TartanGround root
    output_dir    = 'data/training',
    max_trajs     = 300,                   # use 300 trajectories
    frames_per_traj = 8,                   # merge 8 LiDAR frames per traj
    save_interval = 25,                    # print progress every 25
)
```

Terminal -- this takes 1-3 hours for 300 trajectories

```
python build_training_data.py
```

EXPECTED OUTPUT

```
[IMURiskDataset] Building training data from /mnt/tartanground
  Scenes available: 63
  Max trajectories: 300
  Output: data/training

  [  1/300] forest_01/P000:  284,291 pts  mean_risk=0.312  imu_dur=64.0s
  [  2/300] forest_01/P001:  271,804 pts  mean_risk=0.287  imu_dur=58.3s
  ...
  [ 25/300] urban_02/P000:   198,432 pts  mean_risk=0.441  imu_dur=52.1s
  ...
  [300/300] desert_rocky/P017: 310,847 pts  mean_risk=0.583  imu_dur=71.4s

[IMURiskDataset] Done. 300 trajectories -> data/training
  Total points: 78,421,930
  Mean risk across dataset: 0.381
```

Output files in `data/training/` -- one `traj_<scene>_<id>.npz` per trajectory, containing features(N,8), labels(N,), risks(N,)

Phase 5 — Train the Terrain Classifier

Now train a neural terrain classifier on the .npz files you just built. This replaces the heuristic labels with a model that has seen 300 real environments.

11 Create and run the classifier training script

train_classifier.py -- save and run

```
# train_classifier.py
import sys
sys.path.insert(0, '.')
from models.terrain_classifier import DatasetBuilder, Trainer
from terrain.segmenter import TerrainMLP

# Load from the .npz files built in Phase 4
train_ds, val_ds = DatasetBuilder.from_tartanground_npz(
    npz_dir      = 'data/training',
    val_fraction = 0.15,
)
```

```
model   = TerrainMLP()
trainer = Trainer(
    save_path  = 'models/weights/terrain_classifier.pt',
    epochs     = 60,
    batch_size = 2048,
)
trainer.fit(model, train_ds, val_ds)
```

Terminal -- 20-40 min on CPU, 5-10 min on GPU

```
python train_classifier.py
```

EXPECTED OUTPUT

```
[DatasetBuilder] Loading 300 TartanGround .npz files...
[DatasetBuilder] Train: 20,082,440  Val: 3,543,960
  Epoch   5/60  train=0.4821  val=0.4103  mIoU=0.412
  Epoch  10/60  train=0.2984  val=0.2761  mIoU=0.589  <- best
  Epoch  20/60  train=0.1841  val=0.1703  mIoU=0.714  <- best
  Epoch  40/60  train=0.1102  val=0.1088  mIoU=0.803  <- best
  Epoch  60/60  train=0.0891  val=0.0941  mIoU=0.821
Best val loss: 0.0891  ->  models/weights/terrain_classifier.pt
```

Saved to: `models/weights/terrain_classifier.pt`

Phase 6 — Train the Risk Predictor

Train the neural risk estimator. Uses the same .npz files — the risks column was derived from IMU vibration + slip signals in Phase 4.

12 Create and run the risk predictor training script

train_risk.py -- save and run

```
# train_risk.py
import sys, numpy as np
sys.path.insert(0, '.')
from pathlib import Path
from models.risk_predictor import RiskDataset, RiskPredictorNet, RiskTrainer

# Load .npz files
npz_dir = Path('data/training')
files   = sorted(npz_dir.glob('traj_*.npz'))
print(f'Loading {len(files)} files...')
```

```
all_f, all_l, all_r = [], [], []
for fpath in files:
    data = np.load(fpath)
    all_f.append(data['features'])
    all_l.append(data['labels'])
    all_r.append(data['risks'])

X = np.vstack(all_f)
y_l = np.concatenate(all_l)
y_r = np.concatenate(all_r)

perm  = np.random.permutation(len(X))
X, y_l, y_r = X[perm], y_l[perm], y_r[perm]
n_val = int(len(X) * 0.15)

train_ds = RiskDataset(X[n_val:], y_l[n_val:], y_r[n_val:], augment=True)
val_ds   = RiskDataset(X[:n_val],  y_l[:n_val],  y_r[:n_val],  augment=False)

model   = RiskPredictorNet()
trainer = RiskTrainer(
    save_path = 'models/weights/risk_predictor.pt',
    epochs    = 80,
    batch_size= 1024,
)
trainer.fit(model, train_ds, val_ds)
```

Terminal -- 15-30 min

```
python train_risk.py
```

EXPECTED OUTPUT

```
Loading 300 files...
  Epoch  10/80  train=0.02841  val=0.02698  <- best
  Epoch  20/80  train=0.01903  val=0.01841  <- best
  Epoch  40/80  train=0.01102  val=0.01034  <- best
  Epoch  80/80  train=0.00891  val=0.00903
Best val Huber loss: 0.00891  ->  models/weights/risk_predictor.pt
```

Saved to: `models/weights/risk_predictor.pt`

Phase 7 — Full Run With Trained Models

Now run the pathfinder with your trained weights. The models have seen 300 TartanGround environments and will produce much more accurate terrain labels and risk scores than the heuristic mode.

13 Run on one trajectory with trained weights

Terminal -- full run with trained models

```
python main.py \
  --cloud /tmp/merged_cloud.npy \
  --start -16.4 -3.2 0 \
  --goal   60.7 69.3 0 \
  --weights-clf  models/weights/terrain_classifier.pt \
  --weights-risk models/weights/risk_predictor.pt \
  --window 6 \
  --alternatives 3 \
  --save-graph outputs/paths/forest01_P000_graph.pkl \
  --visualize
```

EXPECTED OUTPUT

```
  OK  Loaded 510,324 points from merged_cloud.npy
  OK  After preprocessing: 48,291 pts
  OK  Terrain labels (neural): ground=31,204 obstacle=9,841 water=1,203 ...
  OK  Risk (upper bound, MC): mean=0.441  max=0.921  high-unc=2,104 pts
  OK  Graph: 1,204 nodes  (1,181 traversable)  24,831 edges
  OK  Path found: 28 nodes
      Total dist   : 88.3 m
      Max window risk  : 0.921
  Alternatives: 3 paths found
  Smoothed path: 177 waypoints  arc=88.3m
```

NOTE: The graph is cached to outputs/paths/forest01_P000_graph.pkl. Future runs on this same merged cloud can add --load-graph to skip the rebuild and go straight to planning.

14 Process all trajectories in one scene (batch)

batch_scene.py -- save and run

```
# batch_scene.py  --  run pathfinder on every trajectory in a scene
import sys, types, numpy as np
sys.path.insert(0, '.')
from pathlib import Path
from lidar.tartanground import TartanGroundDataset
from lidar.preprocessor import estimate_bounds
from main import run
```

```
ds      = TartanGroundDataset('/mnt/tartanground')  # <-- your path
scene   = 'forest_01'                               # <-- your scene
out_dir = Path(f'outputs/paths/{scene}')
out_dir.mkdir(parents=True, exist_ok=True)

for traj in ds.trajectories_in_scene(scene):
    cloud = traj.load_lidar_merged(max_frames=8)
    np.save('/tmp/_traj_cloud.npy', cloud)
    bounds = estimate_bounds(cloud)

    args = types.SimpleNamespace(
        cloud='/tmp/_traj_cloud.npy', synthetic=False,
        start=[bounds['x_min']+2, bounds['y_min']+2, 0.0],
        goal= [bounds['x_max']-2, bounds['y_max']-2, 0.0],
        window=5, voxel=2.0, connect=5.5, max_risk=None,
        weights_clf ='models/weights/terrain_classifier.pt',
        weights_risk='models/weights/risk_predictor.pt',
        no_smooth=False,
        save_graph=str(out_dir/f'{traj.traj_id}_graph.pkl'),
        load_graph=None, alternatives=1,
        n_synthetic=50000, visualize=False, save_anim=None,
    )
    try:
        r = run(args)[0]
        print(f'  {traj.traj_id}: {len(r.path)} nodes  {r.total_dist:.1f}m
risk={r.mean_window_risk:.3f}')
    except Exception as e:
        print(f'  {traj.traj_id}: FAILED  {e}')
```

Terminal

```
python batch_scene.py
```

Phase 8 — Train PointNet++ (Optional, GPU Recommended)

PointNet++ processes raw 3D point geometry directly — it learns what rocks, mud, and flat ground look like as shapes, not just as hand-crafted features. Best accuracy but needs more compute.

15 Check for CUDA GPU

Terminal

```
python -c "import torch; print('CUDA:', torch.cuda.is_available())"
```

NOTE: PointNet++ trains fine on CPU but is ~10x slower. If you have a GPU, it will be detected automatically.

16 Train PointNet++ on TartanGround trajectories

train_pointnetpp.py -- save and run

```
# train_pointnetpp.py
import sys, numpy as np
sys.path.insert(0, '.')
from pathlib import Path
from lidar.tartanground import TartanGroundDataset
from lidar.loader import load_point_cloud
from lidar.preprocessor import preprocess
from terrain.feature_extractor import extract_features
from terrain.segmenter import TerrainSegmenter
from models.pointnet_segmenter import PointNetPP, train

ds  = TartanGroundDataset('/mnt/tartanground')  # <-- your path
seg = TerrainSegmenter('models/weights/terrain_classifier.pt')

clouds, labels = [], []
count = 0
for traj in ds.iter_trajectories():
    if count >= 100: break   # 100 trajectories is enough
    try:
        cloud  = preprocess(traj.load_lidar_merged(max_frames=5))
        feats  = extract_features(cloud)
        lbls   = seg.predict(feats)
        clouds.append(cloud)
        labels.append(lbls)
        count += 1
        print(f'  Loaded traj {count}/100: {traj.scene}/{traj.traj_id}')
    except Exception as e:
        print(f'  Skip: {e}')

model = PointNetPP(in_feat_dim=1, num_classes=5)
train(
    model, clouds, labels,
    save_path  = 'models/weights/pointnetpp.pt',
    epochs     = 80,
    chunk_size = 8192,
)
```

```
Terminal -- several hours on CPU, 30-60 min on GPU
python train_pointnetpp.py
```

```
EXPECTED OUTPUT
  Loaded traj 1/100: forest_01/P000
  ...
  Epoch  20/80  loss=0.4103  best=0.4103
  Epoch  80/80  loss=0.1203  best=0.1098
Training complete. Best model saved -> models/weights/pointnetpp.pt
```

Phase 9 — Generate Animation

17 Generate a vehicle simulation GIF

```
Terminal
python main.py \
  --cloud /tmp/merged_cloud.npy \
  --start -16.4 -3.2 0 \
  --goal   60.7 69.3 0 \
  --weights-clf  models/weights/terrain_classifier.pt \
  --weights-risk models/weights/risk_predictor.pt \
  --window 5 \
  --save-anim outputs/robot_path.gif
```

```
EXPECTED OUTPUT
  Smoothed path: 177 waypoints  arc=88.3m
  Animation saved -> outputs/robot_path.gif
```

Appendix A — IMU File Reference

Every trajectory's imu/ folder contains these exact files. Both .npy and .txt versions exist — the code always tries .npy first, .txt as fallback.

File (.npy / .txt)	Shape	Frame	Used for
`acc`	(T, 3)	Body	Vibration risk — deviation from gravity = rough terrain
`acc_nograv`	(T, 3)	Global	Linear accel in world frame — motion dynamics

File (.npy / .txt)	Shape	Frame	Used for
`acc_nograv_body`	(T, 3)	Body	Linear accel body frame — useful for ML features
`gyro`	(T, 3)	Body	Slip risk — angular rate / forward speed = slip score
`pos_global`	(T, 3)	Global	Ground-truth XYZ — maps IMU risk to spatial positions
`vel_global`	(T, 3)	Global	GT velocity world frame
`vel_body`	(T, 3)	Body	Denominator of slip score — speed in robot frame
`ori_global`	(T, 4)	Global	Quaternion [qw,qx,qy,qz] — used for coordinate transforms
`imu_time`	(T,)	--	Timestamps for acc + gyro signals
`cam_time`	(C,)	--	Camera timestamps (lower freq than IMU)
`parameter.yaml`	YAML	--	Sensor specs, gravity vector, calibration

Risk formula used on these signals:

Risk derivation logic

```
vibration = | acc_body_magnitude - 9.81 |          # deviation from gravity
slip       = gyro_magnitude / (vel_body_magnitude + 0.1)
instability= gyro_magnitude

risk = 0.45 * norm(vibration)
     + 0.35 * norm(slip)
     + 0.20 * norm(instability)

# Smoothed over 0.5 second windows to reduce noise
# Mapped back to point cloud via pos_global spatial lookup
```

Appendix B — Troubleshooting

Problem	Fix
`open3d ImportError`	pip install open3d
`yaml ImportError`	pip install pyyaml
`acc.npy not found`	Check imu/ subfolder exists in your trajectory path
`No path found`	Add --max-risk 0.999 to your command

Problem	Fix
`Very slow on large .pcd`	Add --voxel 3.0 to reduce node count
`Binary PCD fails to parse`	Already handled -- code falls back to open3d automatically
`ori_global has 3 columns`	Euler angles detected -- code converts to quaternion automatically
`CUDA out of memory`	Reduce chunk_size to 4096 in train_pointnetpp.py
`Timestamps missing`	imu_time.npy missing -- code synthesises 100Hz timestamps automatically
`pos_global shape wrong`	Check file with: numpy.load('pos_global.npy').shape -- should be (T,3)

Appendix C — All CLI Flags

Flag	Description / default
`--cloud PATH`	Path to .pcd / .ply / .npy / .npz file
`--synthetic`	Use auto-generated terrain for testing
`--start X Y Z`	Start position in metres (required)
`--goal X Y Z`	Goal position in metres (required)
`--window N`	Joint-risk lookahead window [default: 5]
`--voxel FLOAT`	Node voxel cell size metres [default: 2.0]
`--connect FLOAT`	Max edge radius metres [default: 5.5]
`--max-risk FLOAT`	Max joint risk threshold [default: auto]
`--alternatives N`	Number of alternative paths [default: 1]
`--weights-clf PATH`	Terrain classifier weights .pt
`--weights-risk PATH`	Risk predictor weights .pt
`--no-smooth`	Skip spline smoothing
`--save-graph PATH`	Cache graph as .pkl
`--load-graph PATH`	Load cached graph (skip rebuild)
`--visualize`	Open 6-panel matplotlib dashboard
`--save-anim PATH`	Save vehicle animation as .gif

Generating waypoints for simulation:

```
Terminal
Python off_road_navig/run_simulation.py
Python off_road_navig/run_simulation_Astar
```

IMPORTANT:
To generate the dataset that was used in the simulation of the phase 2 of the project which required ROS and SLAM, only run the first command above. That is necessary to generate all the waypoints, to proceed to second phase.
To check for comparison with normal A* algorithm, run the second command and visually compare the difference.